

# CloudChaser EVBs User Manual

## Document Information

|                        |                              |
|------------------------|------------------------------|
| Document number:       | SIVDOC-410959866-145725      |
| Document title:        | CloudChaser EVBs User Manual |
| Revision:              | 1.0                          |
| Author:                | Sivers Wireless              |
| Document type:         |                              |
| Approved by:           |                              |
| Date:                  |                              |
| Confidentiality class: | Internal Only                |

Neither the whole nor any part of the information contained in, nor the product described in, this document may be adapted or reproduced in any form. Distribution outside of Sivers Semiconductors is strictly forbidden.

## Revision History

| Revision | Date                      | Change description |
|----------|---------------------------|--------------------|
| 1.0      | 19 <sup>th</sup> Feb 2025 | First Release      |
|          |                           |                    |
|          |                           |                    |
|          |                           |                    |

## Table of Contents

|     |                                                    |    |
|-----|----------------------------------------------------|----|
| 1.1 | Overview Stampede2731 EVB .....                    | 4  |
| 1.2 | Overview Blueway1721 EVB .....                     | 5  |
| 2.1 | Connection SPI .....                               | 6  |
| 2.2 | Power Supply.....                                  | 6  |
| 3.1 | Stampede2731/Blueway1721 Theory of Operation ..... | 8  |
| 3.2 | Digital Interface Pinout .....                     | 12 |
| 5.1 | SPI Transaction.....                               | 17 |
| 5.2 | SPI Commands .....                                 | 20 |
| 6.1 | One channel .....                                  | 24 |
| 6.2 | Full channel .....                                 | 24 |

## 1 General Overview

### 1.1 Overview Stampede2731 EVB

The Stampede2731 Evaluation board is designed to be a test board for the Stampede2731 TX Satcom Beamforming IC. The EVB contains.

- Power cable with 1, 1.8 and 4.0V supply input
- the interface to PC with USB to SPI circuitry (FTDI device)
- Beam 0 and beam 1 RF input
- RF H Channel 1-4 TX and V Channel 1-4 TX
- Software driver for Windows (download from Sivers Semiconductors sftp)

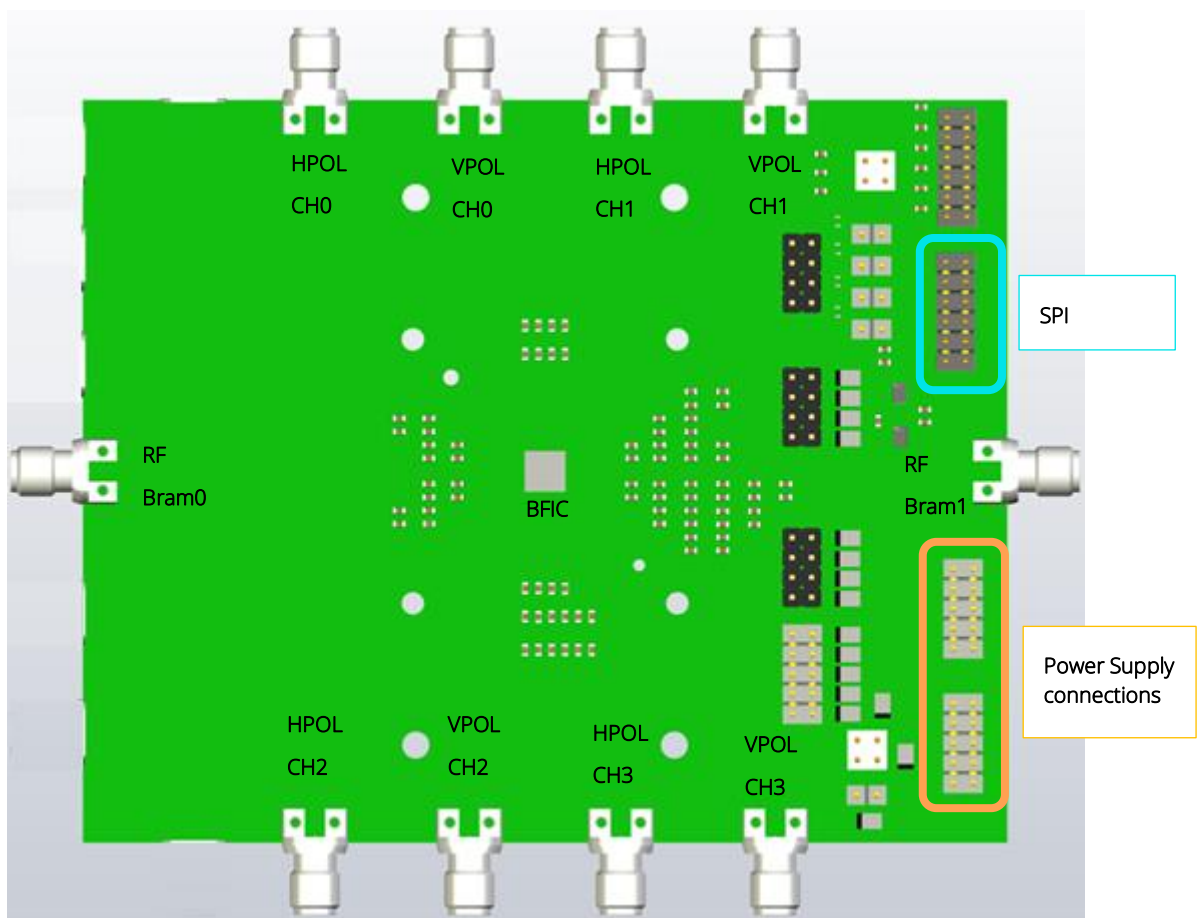


Figure 1: Stampede2731 EVB Outline

## 1.2 Overview Blueway1721 EVB

The Blueway1721 Evaluation board is designed to be a test board for the Blueway1721 RX Satcom Beamforming IC. The EVB contains.

- Power cable with 1, 1.8 and 1.5V supply input
- the interface to PC with USB to SPI circuitry (FTDI device)
- Beam 0 and beam 1 RF output
- RF H Channel 1-4 RX and V Channel 1-4 RX
- Software driver for Windows (download from Sivers Semiconductors sftp)

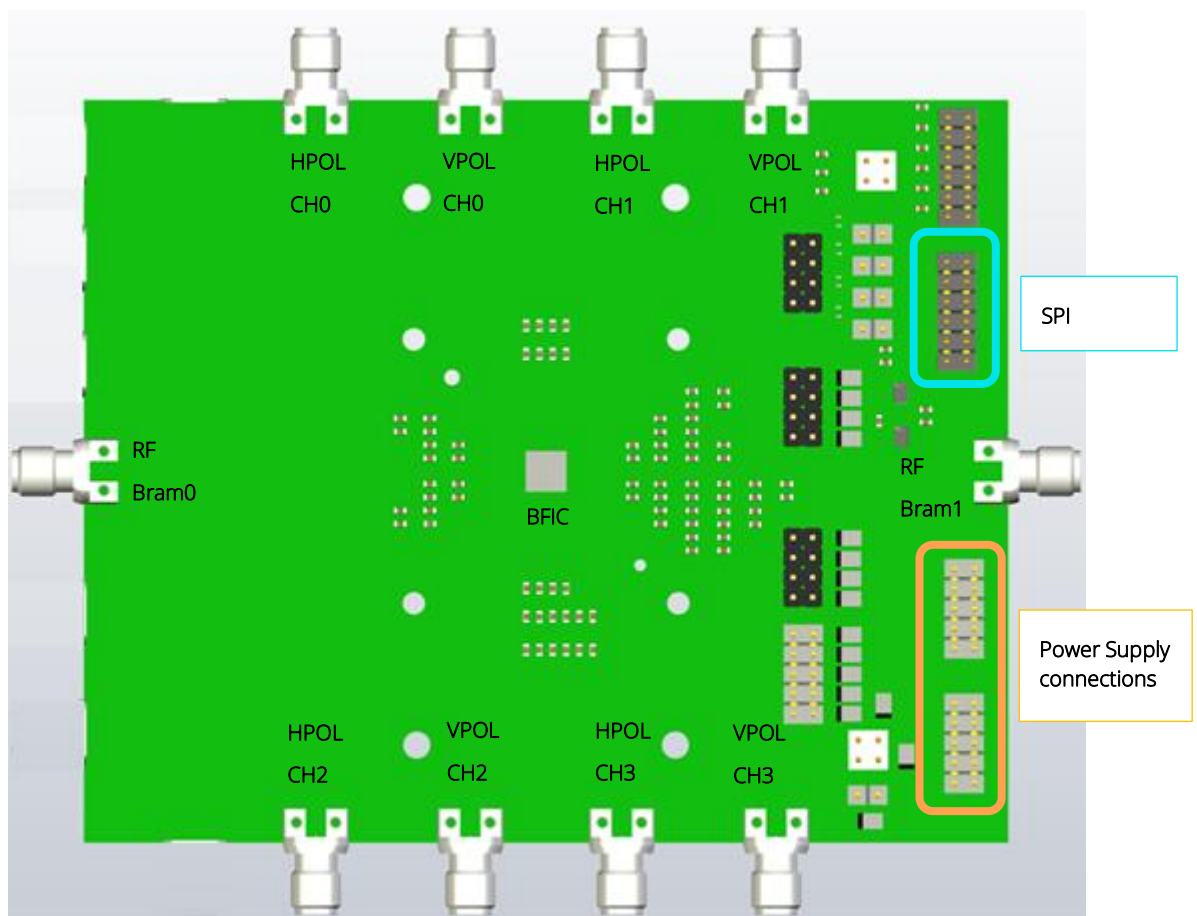
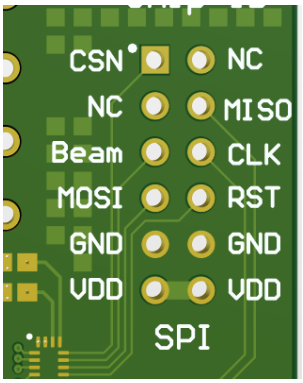


Figure 2: Blueway1721 EVB Outline

## 2 EVB connections

### 2.1 Connection SPI

Pinout of the included usb ftdi dongle

| Description                                                                                                                                                                                                                                                                              | USB dongle connection                                                                                                                                                                                                                                                  | EVB SPI connection                                                                                                                                                      |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <ul style="list-style-type: none"> <li>MOSI is DIN</li> <li>MISO is SPI_DOUT</li> <li>SPI_CSN is Chip Select and goes to CSN</li> <li>RESET_N is reset and goes to RST</li> <li>Black is GND</li> <li>Blue and white are unused on these evb therefore these shall be left NC</li> </ul> |  <p>1 RED 3.3V<br/>2 ORANGE SPI_CLK<br/>3 YELLOW MOSI<br/>4 GREEN MISO<br/>5 BROWN SPI_CSN<br/>6 GREY RESET_N<br/>7 PURPLE BEAM_UPD<br/>8 WHITE TR_SWITCH<br/>9 BLUE<br/>10 BLACK</p> |  <p>CSN NC<br/>NC MISO<br/>Beam CLK<br/>MOSI RST<br/>GND GND<br/>VDD VDD<br/>SPI</p> |

Tabell 1: SPI Connection

### 2.2 Power Supply

The table shows how to connect each board

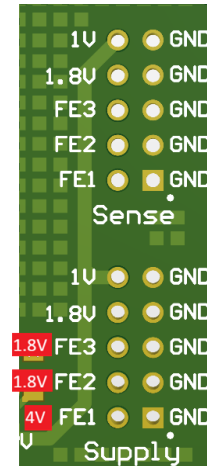
| Connector      | TX - Board | RX - Board |
|----------------|------------|------------|
| 1V DIG         | 1.0V       | 1.0V       |
| 1.8V I/O / ANA | 1.8V       | 1.8V       |
| FE3: ANA       | 1.8V       | 1.5V       |
| FE2: PA / LNA  | 4.0V       | 1.0V       |
| FE1: DRV       | 1.8V       | 1.0V       |
| EFUSE          | GND        | GND        |

Tabell 2: Power Supply Connection

### 2.2.1 TX Power Connection

There is an additional power connector next to the first one called "Sense". This one can be utilized as sense connection in the case of large voltage drops in long cables.

Setup current limiting supplies as per datasheet.

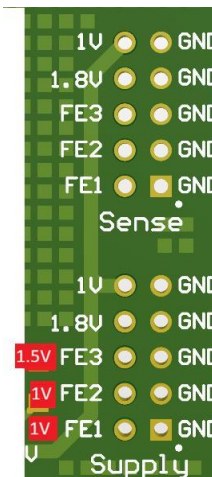


Tabell 3: TX Power Connection

### 2.2.2 RX Power Connection

There is an additional power connector next to the first one called "Sense". This one can be utilized as sense connection in the case of large voltage drops in long cables.

Setup current limiting supplies as per datasheet.



Tabell 4: RX Power Connection

### 3 Functional overview

#### 3.1 Stanpede2731/Blueway1721 Theory of Operation

The STAMPEDE/BLEUWAY device is a beam forming IC with 8 channels of transmit or receive processing grouped into 2 I/O ports as show in the figure below. It supports two independent beamformer phase and gain control for each channel. It includes a common mode gain adjustment for each I/O port. Transmit power and temperature telemetry can be read though the SPI control interface. Embedded efuse can be programmed so that the device will initialize calibration and bias after reset.

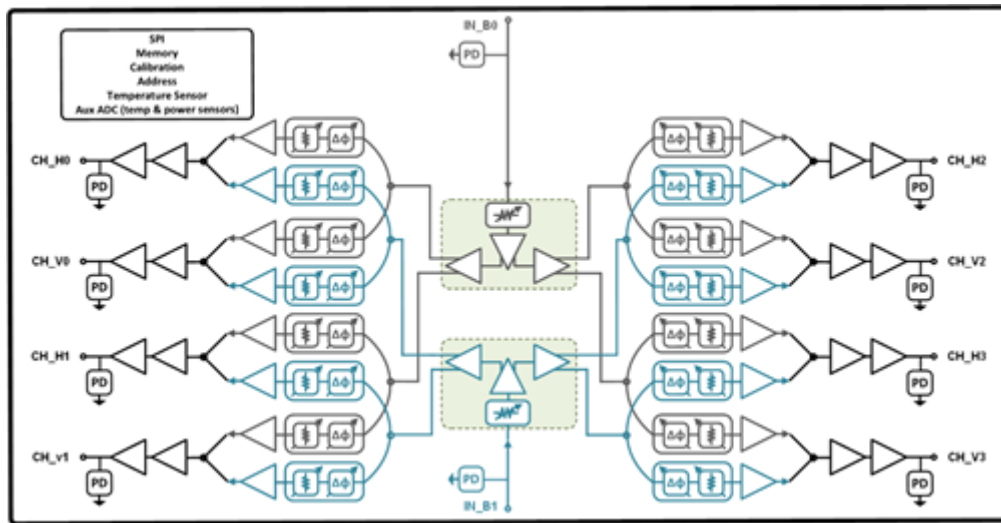


Figure 3: Stampede2731 Block Diagram

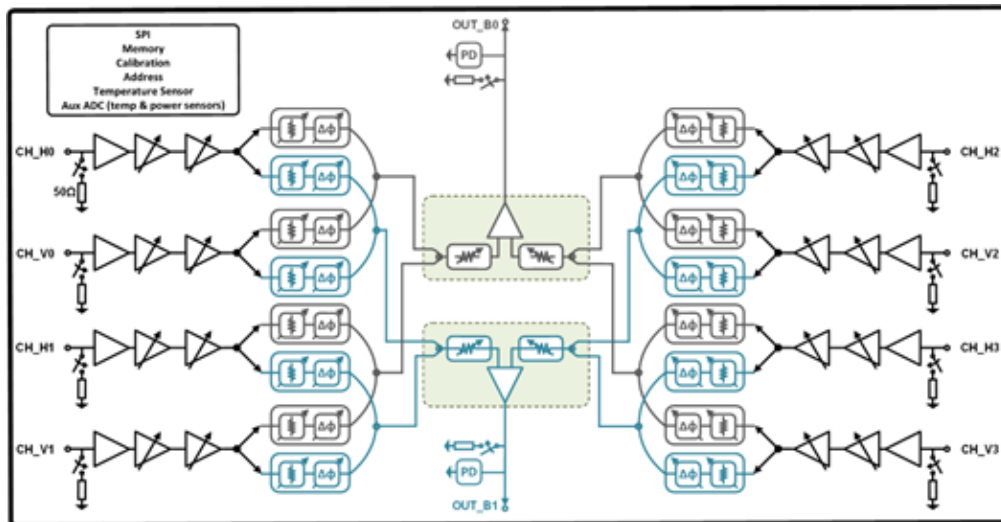


Figure 4: Blueway1721 Block Diagram

The channels are grouped into Quads which have common functionality, and the control registers are organized by Quad. Each Quad supports 2 channels (antenna interfaces) with two beam state per channel. The table below indicated the channel grouping:



| Quad | Channels                            |
|------|-------------------------------------|
| 1    | H0: beam0, beam 1; V0: beam0, beam1 |
| 2    | H1: beam0, beam 1; V1: beam0, beam1 |
| 3    | H2: beam0, beam 1; V2: beam0, beam1 |
| 4    | H3: beam0, beam 1; V3: beam0, beam1 |

Tabell 5: Channel Grouping

The structure of a channel pair in a quad is shown below:

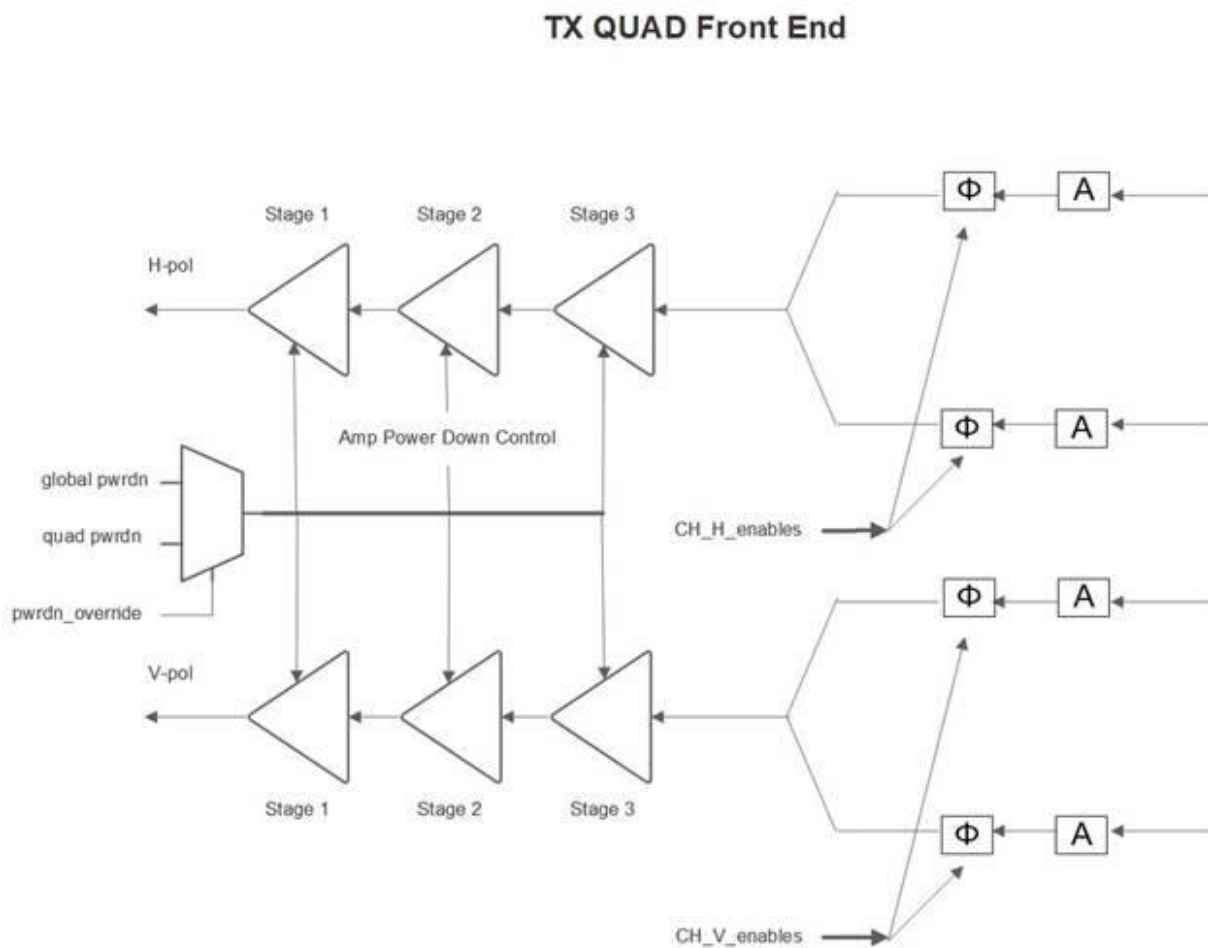


Figure 5: Stampede2731 Quad Block

## RX QUAD Front End

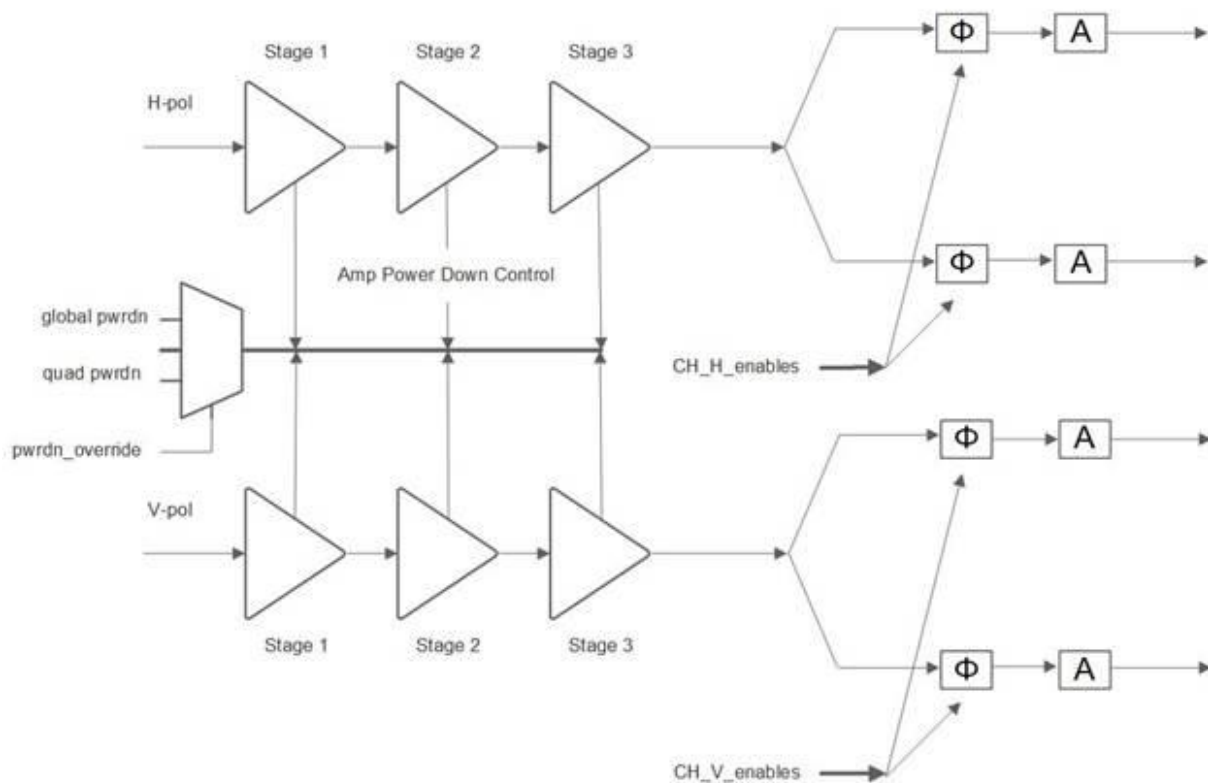


Figure 6: Blueway1721 Quad Block

For each Quad there are enable bits, described below, which can be used to enable or disable the amps and switches for each channel. When the amps and switches are enabled (bit is set to 1) their ON or OFF state will be determined by the EN\_STATE. And setting an enable bit to 0 will force the particular amp or switch OFF independent of the enable state (EN\_STATE) disabling the function, and, in the case of the amplifiers, turning off the power. After reset, all amps and switches default to disabled.

Additionally, there are auto-power down operations for the BEAM and AMP enables. A particular BEAM will be disabled for both H and V if the beam attenuator is set to its maximum value of 127. If all the beams being associated with a PA or LNA are disabled, then that amplifier will be powered down as well. The logic for these operations is shown in the figure below:

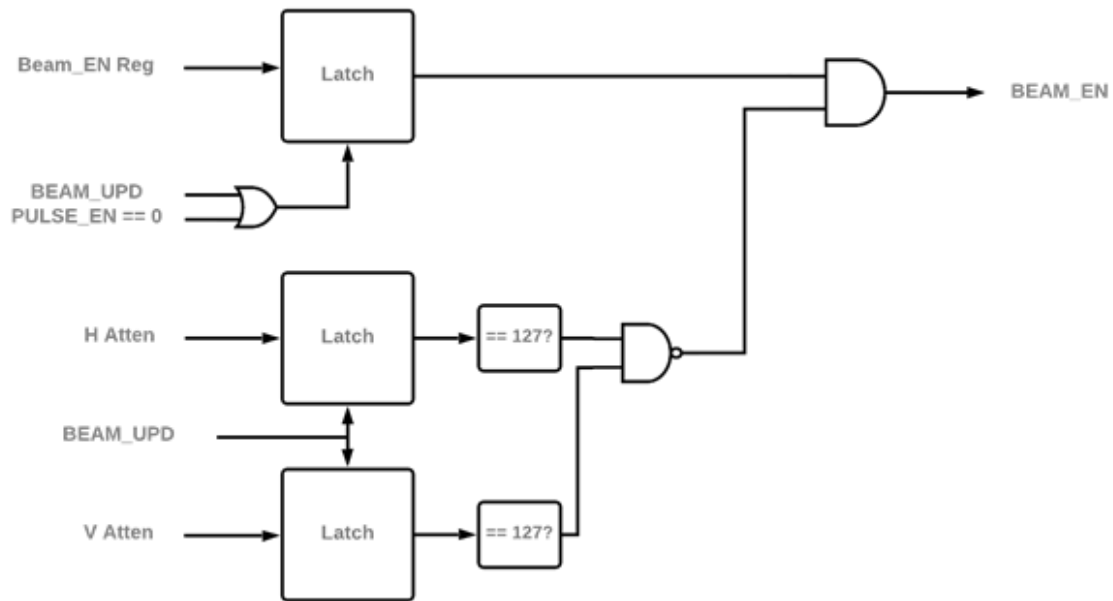


Figure 7: Beam auto disable

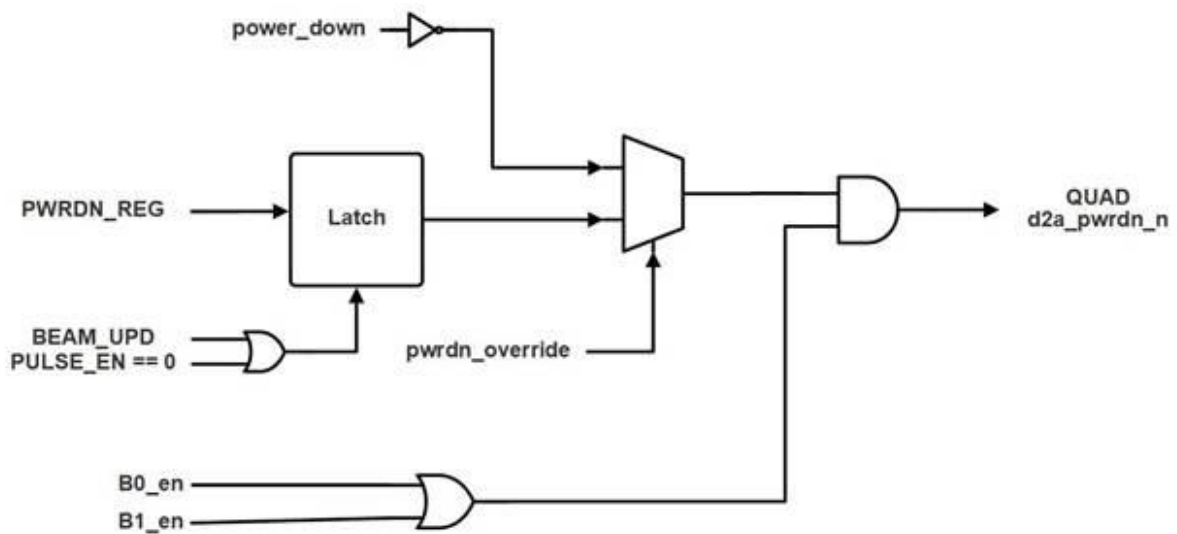


Figure 8: Amp auto power down

Each I/O port has a similar set of enables called Common enables, described below, which allow for the IO amps and switches to be disabled for each beam. These are disabled by default after reset. For a full beam path to operate at least 1 Quad beam and the Common amps and switches must be enabled.

Each channel can have its EN\_STATE set independently. This can either be by enabling global control of the EN\_STATE by the Global\_power\_down bit or overriding the control through a programmable register, described below. In automatic mode, a value of 0 on the Global\_power\_down bit will configure the channels for OFF mode (i.e. turn OFF amps), and a value of 1 will configure the

channels for ON mode (i.e. turn ON enabled amps). In override mode, the bits of the EN\_OVERRIDE register determine which enabled amps are ON for each channel. The override operation is also set through for each channel through the EN\_OVERRIDE register. After reset, the default operation of EN\_STATE is override for all channels. The beam based auto power down will override either of these enable methods.

Each channel has multiple amplifier stages. Each amplifier stage has a bias current control which may be programmed through the SPI interface. The bias control is a 6-bit number with 0 being the minimum current and 63 being the maximum. There are also bias controls for the IO amplifiers and active combiners/splitters, which can be similarly set. After reset, the bias values are loaded from efuse to their default state which will be 0 before the efuse is programmed.

The beamforming phase and attenuation applied to a channel are selected from the large beam table stored on chip. This table can be thought of as a 4-channel x 1024-line memory with 16 bits of phase and attenuation control per channel (i.e. 64 bits x 1024 lines). A line pointer into this memory is used to select which set of beamforming coefficients are applied for each beam and channel. There are sixteen independent pointers which select beam0, beam1 for each of 8 channels (H and V). The pointers can select different lines within the memory or lines can be reused between beams and channels. Each pointer has a shadow register which is written by a SPI transaction and then applied to index the memory when the BEAM\_UPDATE pin is pulsed high. If the BEAM\_UPDATE pin is held at 1 then the writing of a pointer is applied immediately at the end of the SPI transaction. After reset, the beam pointers default to index 0.

Calibration of phase and gain is available for each beam on a per channel basis. There is a look up table for each channel which is loaded from efuse. Each look up table contains parameters for a calibration function to map from linear phase command to the optimal control word output for the phase shifter control.

## 3.2 Digital Interface Pinout

The figure below shows the top level of the SPI interface block. The pins are defined below

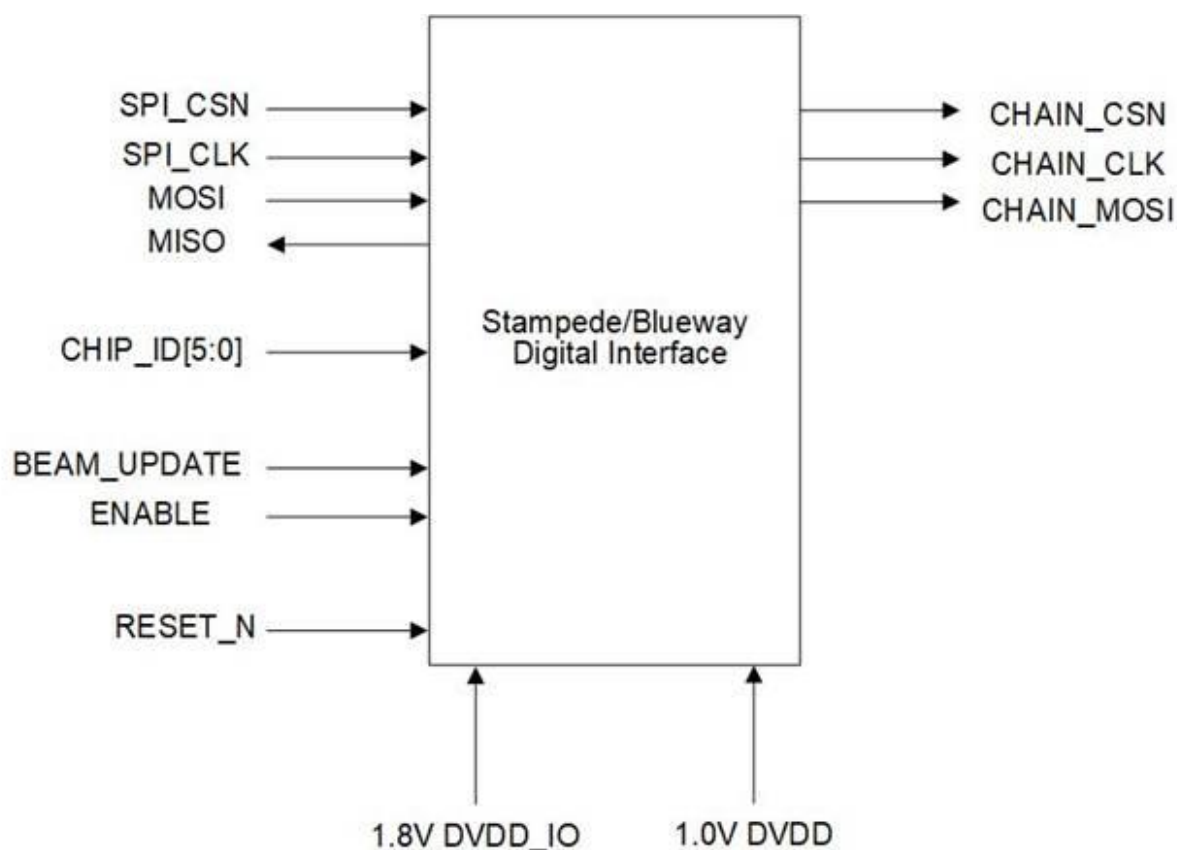


Figure 9: Stampede2731/Blueway1721 Digital Interface

| Pin Name    | Direction | Connection         | Function                                   |
|-------------|-----------|--------------------|--------------------------------------------|
| SPI_CSN     | In        | external pad       | SPI chip select, active low                |
| SPI_CLK     | In        | external pad       | SPI clock                                  |
| SPI_MOSI    | In        | external pad       | SPI data master out slave in               |
| SPI_MISO    | out       | external tri-state | SPI data slave out master in               |
| RESET_N     | In        | external pad       | block reset, active low                    |
| BEAM_UPDATE | In        | external pad       | update beam index latch                    |
| CHAIN_CSN   | out       | external pad       | Output buffer for SPI_CSN for daisy chain  |
| CHAIN_CLK   | out       | external pad       | Output buffer for SPI_CLK for daisy chain  |
| CHAIN_DATA  | out       | external pad       | Output buffer for SPI_MOSI for daisy chain |

|                     |        |              |                                                        |
|---------------------|--------|--------------|--------------------------------------------------------|
| CHIP_ID[0]/ENUM_IN  | In     | external pad | CHIP_ID bit 0 multiplexed with Enum daisy chain input  |
| CHIP_ID[1]/ENUM_OUT | In/Out | External pad | CHIP_ID bit 1 multiplexed with Enum daisy chain output |
| CHIP_ID[5:2]        | In     | external pad | CHIP_ID bits 2 to 5                                    |

Tabell 6: Digital Pinout

## Array Interconnect

The STAMPEDE/BLEWAY is designed to be used in an array with a single SPI interface addressing all the chips in the array. The intended interconnect is shown below in Figure 8 and Figure 9. In Figure 8 the SPI\_CLK, SPI\_CSN, SPI\_MOSI, SPI\_MISO, BEAM\_UPDATE, and RESET\_N are all wired in common to all the array devices. SPI\_MISO is a tri-state buffer which is only active when a READ command is addressed to a particular device. This permits the devices to share the return bus.

Alternatively, as shown in Figure 9 the chips can be connected in a daisy chain configuration where the CHAIN outputs of one chip drive the inputs to the next chip and leads to simplified board routing. In this configuration the SPI\_MISO is not used, instead during a read operation the output data is muxed onto the CHAIN\_DATA output and propagates through the daisy chain. The CHAIN\_DATA output of the last element in the chain should be connected to the master data input port.

The CHIP\_ID lines are used to set individual address for the devices within the array and the SPI transaction allows for individual addressing to this address (Figure 8). Optionally the CHIP\_ID can be enumerated by the devices through a daisy chain connection from CHIP\_ID[1] of a device to CHIP\_ID[0] of the next device in the chain. (Figure 9).

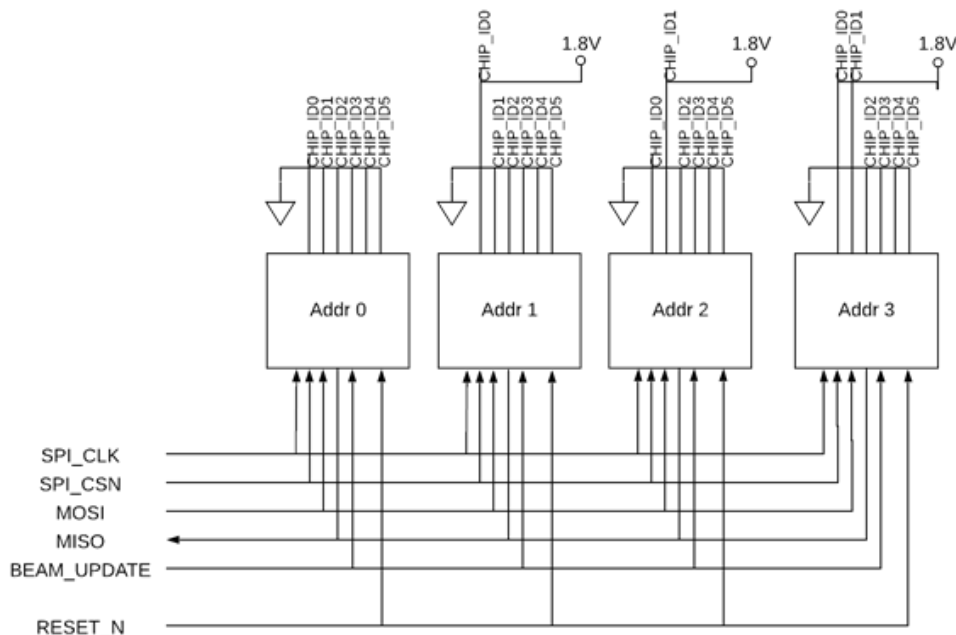


Figure 10: Array Interconnect

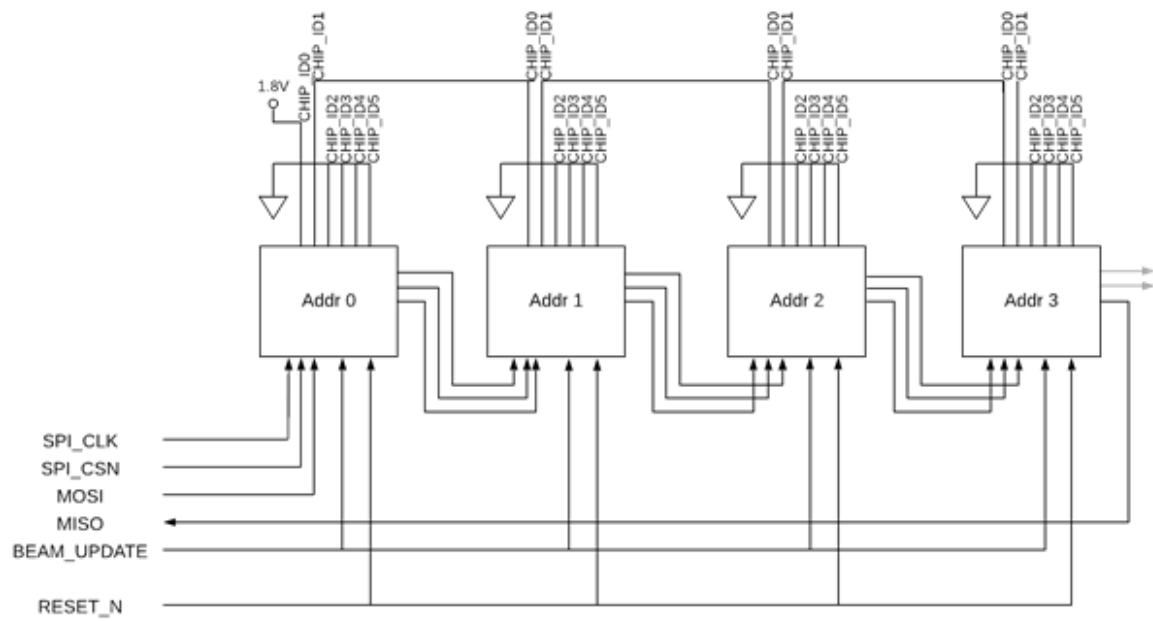
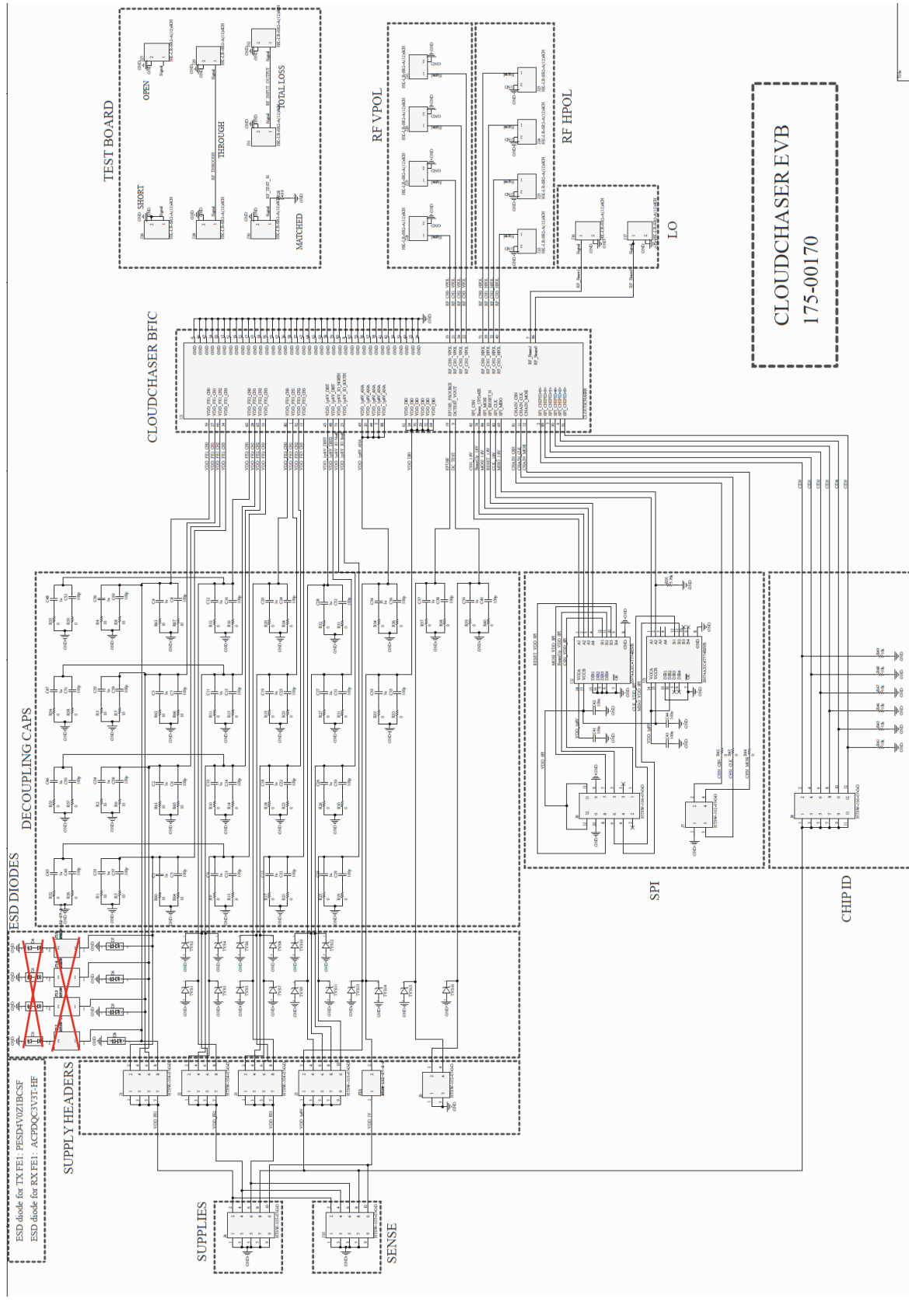


Figure 11: Array Interconnect with Enumeration

## 4 EVB Schematic





## 5 SPI Interface Description

The SPI interface is a serial programming interface for the STAMPEDE/BUEWAY beamforming RF ICs. It supports reading and writing configuration, status, and control registers through a 1-bit serial data interface.

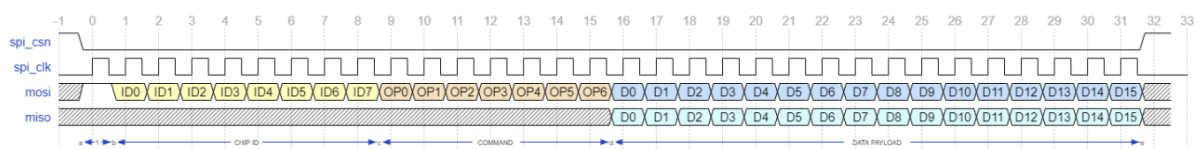
Features:

- Read or Write of 16-bit registers with a single 32-bit transaction.
- Continuous sequential reading or writing of registers through payload extension.
- Continuous sequential writing of beam index pointers through payload extension.
- 100 MHz serial clock.
- 8-bit CHIP ID to allow for individual addressing of up to 255 devices on a single data line.
- Auto-enumeration or pin programming of CHIP ID
- Broadcast addressing to write all devices simultaneously.
- Indirect addressing of registers through an address pointer.
- Auto-increment of address pointer for simplified accessing of sequential registers.
- 2 beams per channel
- Asynchronous beam table index update.
- Enable state machine to manage LNA power and switch state.
- ADC capture control for sampling power detector and temp sensor
- ADC clock control – programmable divider and enable for ADC clock.
- Efuse based calibration of bias currents, temp sensors, power detectors, and RTPS.
- 32-bit unique ID
- Efuse programming and readback interface
- Output buffers for Daisy chain through connectivity of devices

### 5.1 SPI Transaction

The SPI interface uses an atomic 32 or 56-bit transaction. The transaction is initiated by the chip-select line going low, followed by 32 or 56 clock rising edges, and terminated by the chip-select line rising. 56-bit transactions are only used with Unique ID addressing and are restricted to programming the CHIP ID.

The general structure is shown in the Table below



The primary transaction uses a SHORT\_ID (8-bits) to address a chip and is 32-bits long. There is also specialized command for programming the CHIP\_ID that uses the device Unique ID for addressing and is 56-bit long. The contents of the SPI transaction are given in the table below. Data is sent in LSB first i.e. bit 0 corresponds to the first clock of the transaction.

| Bits  | Name           | Description                                                   |
|-------|----------------|---------------------------------------------------------------|
| 0     | ID Type        | 1 indicates 8 bits CHIP ID addressing                         |
| 1-8   | CHIP ID [0:31] | Chip identifier programmed by tying external pins high or low |
| 9-15  | Command [0:6]  | See command table                                             |
| 16-31 | Payload [0:15] | 16-bit data payload                                           |

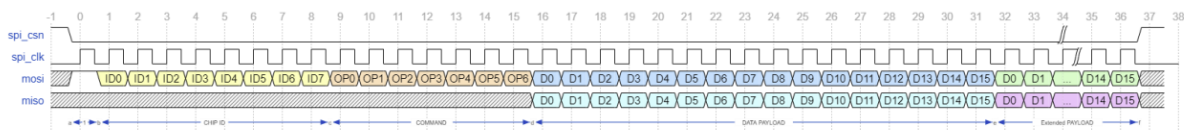
**Tabell 7: 32-bit addressing**

| Bits  | Name           | Description                             |
|-------|----------------|-----------------------------------------|
| 0     | ID Type        | 0 indicates 32-bit Unique ID addressing |
| 1-32  | CHIP ID [0:31] | Chip identifier is 32-bit Unique ID     |
| 33-39 | Command [0:6]  | See command table                       |
| 40-55 | Payload [0:15] | 16-bit data payload                     |

**Tabell 8: 56-bit addressing**

### 5.1.1 Extended Transaction

The interface supports extended operations for read and write if the SPI\_CSN remains low for additional clock cycles after the completion of the basic transaction. In the mode of operation the CHIP\_ID and Command are retained from the start of the transaction and each subsequent set of 16 bits is treated as another payload. Thus for a write command, each subsequent payload would write into a register and cause the load pointer to auto increment.



### 5.1.2 SPI Special Functions

The SPI interface has additional control lines and internal control functions to implement special features of the interface.

RESET\_N is an active low reset signal which puts the SPI interface into a known state. The address pointer, and both beam set registers are all set to 0. The CNTL bits are initialized to a default state as are the bits of Beam Set 0. All other beam set bits are defaulted to 0. Reset will also initiate an

EFZ\_SENSE\_ALL and the SPI\_CLK must be active after RESET\_N is released to allow the EFUSE to be read

and the calibration bits initialized.

**BEAM\_UPDATE:** this line will load the beam coefficient set into a latch which is used to select the Beam Set output to the analog circuits when it is pulsed high. This line controls a transparent latch so if it is held high then a write to a beam set register will take effect immediately. This line allows for precise control of the beam switching time. If this line is held high the beam former parameters will be updated when a beam set write is completed. This line can also be used to control the update of certain enables, gains controls, and power down bits. These registers will not be updated unless BEAM\_UPDATE is high when the associated PULSE\_EN bit is set in the control register.

**ADC sampling:** the ADCs associated with the power detectors and the temperature detectors will only execute a conversion in response to ADC\_CAPTURE command. The ADC value will be updated into the

ADC register at the end of the transaction and can be read with a subsequent transaction.

**CHIP\_ID:** there are 6 external pins which are used to set the 6 LSBs of the SHORT ID for any device. Upon reset the device SHORT ID will be set based on these pins with the 2 MSBs set to 0. This ID can be overwritten by using the devices Unique ID with a WRITE\_SHORT\_ID command. Additionally, the device enumeration process can be used to change the SHORT\_ID. A SPI transaction will only be executed if the 8 ID bits of the transaction match the SHORT ID setting.

**BROADCAST ID:** the SHORT ID of “11111111” is a broadcast id and all chips will respond to a transaction beginning with this value. This is intended to update the beam set or enable register for all chips in an array without addressing them individually, but it also applies to other register writes. Reads are not supported from the broadcast ID to avoid drive conflict of multiple chips in a wired-or connection of SPI\_MISO.

**Calibration and Efuse:** The SPI interface includes many bits for calibration and static control, such as bias settings for the many amplifiers. These bits are addressable via SPI read and write transactions. They can also be initialized from an embedded efuse block. It is intended that the optimal values be programmed into the efuse during manufacturing test and the loaded into the calibration bits upon reset. The SPI interface also includes a mechanism for programming and reading back the efuse data

## 5.2 SPI Commands

The interface uses the 7 bit command word to determine how the payload data is used. The commands are summarized in the table below.

| Command            | Value      | Description                                    |
|--------------------|------------|------------------------------------------------|
| LOAD_POINTER       | 7'b0000000 | Load read/write address pointer from payload   |
| READ_REGISTER      | 7'b0000001 | Read from address pointer location             |
| WRITE_REGISTER     | 7'b0000010 | Write to address pointer location              |
| WRITE_H0_BEAM0_IDX | 7'b0100000 | Write beam set pointer for beam0 H0            |
| WRITE_V0_BEAM0_IDX | 7'b0100001 | Write beam set pointer for beam0 V0            |
| WRITE_H1_BEAM0_IDX | 7'b0100010 | Write beam set pointer for beam0 H1            |
| WRITE_V1_BEAM0_IDX | 7'b0100011 | Write beam set pointer for beam0 V1            |
| WRITE_H2_BEAM0_IDX | 7'b0100100 | Write beam set pointer for beam0 H2            |
| WRITE_V2_BEAM0_IDX | 7'b0100101 | Write beam set pointer for beam0 V2            |
| WRITE_H3_BEAM0_IDX | 7'b0100110 | Write beam set pointer for beam0 H3            |
| WRITE_V3_BEAM0_IDX | 7'b0100111 | Write beam set pointer for beam0 V3            |
| WRITE_H0_BEAM1_IDX | 7'b0101000 | Write beam set pointer for beam1 H0            |
| WRITE_V0_BEAM1_IDX | 7'b0101001 | Write beam set pointer for beam1 V0            |
| WRITE_H1_BEAM1_IDX | 7'b0101010 | Write beam set pointer for beam1 H1            |
| WRITE_V1_BEAM1_IDX | 7'b0101011 | Write beam set pointer for beam1 V1            |
| WRITE_H2_BEAM1_IDX | 7'b0101100 | Write beam set pointer for beam1 H2            |
| WRITE_V2_BEAM1_IDX | 7'b0101101 | Write beam set pointer for beam1 V2            |
| WRITE_H3_BEAM1_IDX | 7'b0101110 | Write beam set pointer for beam1 H3            |
| WRITE_V3_BEAM1_IDX | 7'b0101111 | Write beam set pointer for beam1 V3            |
| ADC_CAPTURE        | 7'b0000100 | Initiate ADC sampling                          |
| INIT_ENUM          | 7'b0000101 | Initiate ID enumeration                        |
| SOFT_RESET_CMD     | 7'b0000111 | Reset all writable registers to default values |
| WRITE_SHORT_ID     | 7'b0001000 | Write short CHIP ID                            |

| Command           | Value      | Description                                                               |
|-------------------|------------|---------------------------------------------------------------------------|
| EFZ_ROW_SET       | 7'b1000000 | Set row pointer for efuse sensing or Programming                          |
| EFZ_SET_DATA_LOW  | 7'b1000001 | Set low 16 bits of efuse column data for Programming                      |
| EFZ_SET_DATA_HIGH | 7'b1000010 | Set high 16 bits of efuse column data for Programming                     |
| EFZ_F_VAL         | 7'b1000100 | Set the F-value for sense resistance                                      |
| EFZ_PROG_TIME     | 7'b1000110 | Set the programming timer value                                           |
| EFZ_SET_CHAR      | 7'b1000111 | Set the state of the characterization control bits<br>GATEC, MIMICN, TIME |
| EFZ_PROG_ROW      | 7'b1100000 | Program the current row with column data                                  |
| EFZ_SENSE_ROW     | 7'b1001000 | Sense the current row                                                     |
| EFZ_SENSE_ALL     | 7'b1001001 | Read the entire contents of effuse                                        |

**Tabell 9: SPI Command**

Read and write access to the address space is by indirect addressing through an address pointer register.

To access a specific location this register must be programmed prior to the

**READ\_REGISTER/WRITE\_REGISTER** command via the **LOAD\_POINTER** command. The value stored in the pointer will auto increment with each **READ\_REGISTER/WRITE\_REGISTER** command to facilitate sequential access without the need for additional **LOAD\_POINTER** commands. This pointer will wrap

around to 0 after the last address is accessed (0x11FF) and is initialized to 0 by reset.

Loading the address pointer (**LOAD\_POINTER** command = 0x00) will load the 13 LSBs of the payload into the read/write set register. No other registers are updated.

Loading the one of the beam set registers (**WRITE\_\*\_BEAM\_IDX** command = 0x20 – 0x37) will load the 10 LSBs of the payload into a shadow register to select the beam coefficient set output to the beam former. The actual output is not updated until the BEAM\_UPDATE signal is pulsed as described below. There are twenty-four beam set registers which are loaded independently.

Writing a register (**WRITE\_REGISTER** command = 0x02) will load the 16-bit payload into the address pointed to by the address pointer. The address pointer is incremented by one and will wrap around to 0 if it is currently at the highest address (0x11FF). The writing of the register and the increment of the pointer occur on the falling edge of the last clock of the transaction. Note many registers are not a full 16-bit value and the extra bits are not implemented.

Reading a register (**READ\_REGISTER** command = 0x01) will load the register currently pointed to by the address pointer into an output shift register, whose bits are shifted out on SPI\_DOUT coincident with the payload bits during a transaction. The address pointer is incremented by one and will wrap around to 0 if it is currently at the highest address. SPI\_MISO will be held in a high-Z state until the active payload portion of the read command.

The temperature sensor and power detector ADC updates will be initiated by the **ADC\_CAPTURE** command (command = 0x08). The process will start on the 16th clock of the transaction with the sampling of data and the ADC values will be update  $25 \cdot 2^{\text{ADC\_CLK\_DIV}}$  cycles later. The SPI\_CLK should continue to be active during the ADC capture process although the SPI\_CSN does not need to remain active. The payload is not used for this command.

The **INIT\_ENUM** command (command = 0x09) places the device into a state in which it will perform an enumeration operation on the next SPI\_CSN transaction. The enumeration process is described in more detail below. The payload value is not used by this command.

**SOFT\_RESET** (command = 0x0F) will reset all the registers to their default values. The payload is not used by this command.

**WRITE\_SHORT\_ID** (command = 0x10) uses unique ID addressing (56-bit transaction) to overwrite the CHIP\_ID value. The 8 LSBs of the payload are loaded into the device CHIP\_ID at the completion of this command.

The **EFZ\_ROW\_SET** command (command = 0x40) sets the row pointer for efuse sensing or programming.

This is a 4-bit number loaded from the 4 LSBs of the payload.

The **EFZ\_SET\_DATA\_LOW** command (command = 0x41) will set the lower 16 bits of the efuse column programming register from the payload. During programming, the column locations with a 1 in current row will be blown.

The **EFZ\_SET\_DATA\_HIGH** command (command = 0x42) will set the upper 16 bits of the efuse column programming register from the payload. During programming, the column locations with a 1 in current row will be blown.

The **EFZ\_F\_VAL** command (command = 0x44) will load the 6-bit F-value register for the efuse from the LSBs of the payload. The F-value controls the resistance threshold of the sense amps during efuse readback.

The **EFZ\_PROG\_TIME** command (command = 0x46) will load the 11 LSBs of the payload into a programming timer. The timer determines how long the PROG\_P signal is applied for each column during a programming operation.

The **EFZ\_SET\_CHAR** command (command = 0x47) sets signal used for characterization testing of the efuse IP. The bits are load from the 5 LSBs as mapped below

|           |              |
|-----------|--------------|
| GATEC     | Payload[0]   |
| MIMICN    | Payload[1]   |
| TIME[0:2] | Payload[4:2] |

The **EFZ\_PROG\_ROW** command (command = 0x60) initiates the efuse programming sequence. This is described in further detail below. The SPI\_CLK must remain active for the entire programming sequence although the SPI\_CSN does not need to remain active after the transaction has initiated programming.

The **EFZ\_SENSE\_ROW** command (command 0x48) will read the efuse content for the row pointed to by the current row pointer. The SPI\_CLK must remain active during the entire sense process although the SPI\_CSN does not need to remain active after the transaction has initiated sensing.

The **EFZ\_SENSE\_ALL** command (command 0x49) will read the entire contents of the efuse cycling through rows 0 to 15. The SPI\_CLK must remain active during the entire sense process although the SPI\_CSN does not need to remain active after the transaction has initiated sensing.

## 6 Test Setup

### 6.1 One channel

See example setup script in software release directory.

### 6.2 Full channel

See example setup script in software release directory..